

Data Lakehouse for Time Series Data: A Systematic Literature Review

Matthias Pohl^{1,2}, Nathira Dharindri Wijemanne^{1,2}, Daniel Staegemann², Christian Haertel², Christian Daase², Dirk Dreschel², Damanpreet Singh Walia², Arne Osterthun¹, Joshua Reibert¹, Klaus Turowski²

¹German Aerospace Center (DLR), Institute of Data Science, Jena, Germany

²Faculty of Computer Science, Otto von Guericke University, Magdeburg, Germany

firstname.lastname@ovgu.de, firstname.lastname@dlr.de

Abstract—As data continues to grow exponentially, the fields of data management and analytics must evolve to ensure efficient data ingestion, knowledge extraction, and scalability. The Data Lakehouse architecture, which combines the best features of Data Warehouses and Data Lakes, has emerged as a potential solution. However, to fully leverage the capabilities of Data Lakehouses for time series data, it is crucial to understand the unique challenges and opportunities they present. This literature review examines proposed Data Lakehouse architectures specifically for time series data, exploring their implementation, the software technologies used, and potential real-world applications. The focus is on comparing these architectures to identify the most suitable technologies for similar implementations. Through an in-depth analysis, this study emphasizes the importance of optimizing configurations to enhance system performance and scalability, particularly for data analysis and artificial intelligence (AI) workloads.

Index Terms—Data Lakehouse, Time Series Data, Temporal Data, Virtual Data Warehouse, Literature Review

I. INTRODUCTION

In today's digital age, big data is fundamental for decision-making, innovation, and understanding [1]. As big data continues to grow, it becomes essential for data management and analytics to evolve to provide high scalability and performance [2], [3]. The Data Lake emerged as a centralized repository that allows for the storage of both structured and unstructured data, addressing the limitations of previous systems, which often faced high costs and user loads. However, Data Lakes come with their own set of challenges, including managing schema evolution [4], as well as ensuring data governance and quality [5]. Consequently, the need for the Data Lakehouse architecture, particularly for time series data, has become increasingly important and widespread.

The Data Lakehouse combines the management and performance features of traditional Database Management Systems (DBMS) with the capabilities of Data Warehouses and Data Lakes, creating a unified platform for storing, managing, and analyzing large volumes of data [6]. This approach is particularly advantageous for managing time series data, which is essential for analysis and predictive tasks across various domains. However, since the Data Lakehouse is a relatively new concept, there are several challenges associated with implementing its architecture to ensure data quality [7].

To fully utilize the potential of time series data in Data Lakehouses, a deep understanding of its unique challenges and

opportunities is essential. Therefore, a thorough Systematic Literature Review (SLR) is conducted to identify use cases for time series data in Data Lakehouses, examining different storage methods, various implementation techniques, and the challenges associated with these implementations. Additionally, the review will explore potential solutions to overcome these challenges in the future. This literature review aims to answer the following research questions,

- **RQ1:** What is the most suitable data lakehouse architecture for handling time series data?
- **RQ2:** What are the common use cases of Data Lakehouses for time series data storage in different domains?
- **RQ3:** Which software is utilized in the implementation of the data lakehouse architectures for time series data?

This literature review synthesizes existing knowledge and best practices regarding Data Lakehouses, highlighting their implementations while identifying associated challenges and limitations. It also examines the software used in this architecture. The paper is organized as follows: The "Methodology" section provides an overview of the systematic approach. The "Literature Review" categorizes and discusses relevant research articles. Finally, the "Discussion" addresses validity threats and suggests future research directions, while the "Conclusions" summarizes the findings.

II. METHODOLOGY

This section outlines the systematic approach utilized to identify, select, and analyze relevant sources necessary for a systematic literature review (SLR) [8]. According to [8], the review process consists of three principal stages: planning, conducting, and reporting.

The development of the review protocol starts with identifying the research questions that the review seeks to address. Following this, a search strategy is defined, encompassing the identification of keywords, search terms, appropriate repositories, and inclusion/exclusion criteria. Once the search strategy is established, relevant literature is retrieved, analyzed, and reviewed. The search strategy implemented to refine the literature for consideration will be discussed in further detail in this chapter.

This review utilized a thorough search strategy that encompassed databases, academic journals, conference proceedings, and grey literature repositories. The research focused on

several key databases: Scopus, IEEE Xplore, ACM Digital Library, Web of Science, and Google Scholar. The first four were chosen for their standing as leading academic research resources in the field of Computer Science, especially for journal and conference articles. Google Scholar was included to help identify relevant grey literature.

A blend of keywords and search terms associated with data lakehouse architecture, along with terms pertinent to time series data, was combined using Boolean operators to enhance the search results. Terms such as "financial series," "economic data," and "sensor data" were incorporated to uncover use cases of Data Lakehouses across these areas. The initial search string utilized was as follows,

("lakehouse" OR ("data lake" AND ("data virtualization" OR "data virtualisation")) OR "virtual data warehouse" AND ("time series" OR "longitudinal data" OR "temporal data" OR "economic data" OR "financial series" or "sensor data")

The initial search concluded in June 2024 and resulted in 330 research items. The inclusion and exclusion criteria for a systematic literature review (SLR) are a set of predetermined rules that determine the eligibility of studies for inclusion. To ensure the quality and accuracy of the selected research, the review prioritized peer-reviewed articles, conference papers, and reports. A detailed list of these criteria can be found in Table I.

TABLE I
INCLUSION AND EXCLUSION CRITERIA OF LIT. REVIEW

Inclusion	Exclusion
- Published in a journal, conference, or is an organizational case study - Literature relevant to architectures, software, and use cases of Data Lakehouses or Virtual Data Warehouses	- Published in a language other than English - Paper is a book or a book chapter - Literature is not related to data storage of time series data

A. Literature Screening

After the initial search, a screening process was established to identify and assess the eligibility of potential literature. First, any duplicate entries in the databases were removed. Screening then began with a review of the titles to identify articles related to the storage of time series data. Research articles that were not relevant to data management or storage were excluded. Next, the remaining articles were further analyzed by reviewing their abstracts to ensure they aligned with the research objectives.

Full-text articles were retrieved and reviewed to assess their suitability for inclusion based on predefined criteria. Those articles that met these criteria underwent a thorough evaluation to determine their relevance and contribution to the literature on data lakehouse architecture. Subsequently, the selected

research items were analyzed in detail and categorized into four areas: "Architecture," "Use Cases," "Software," and "Data Schema." At this stage, key findings and insights pertinent to the research question were identified.

The Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) statement was developed to ensure transparent and comprehensive reporting of systematic literature reviews (SLRs) [9]. This statement was adhered to during the screening process of the research articles. The flow diagram depicting this process is provided in Figure 1.

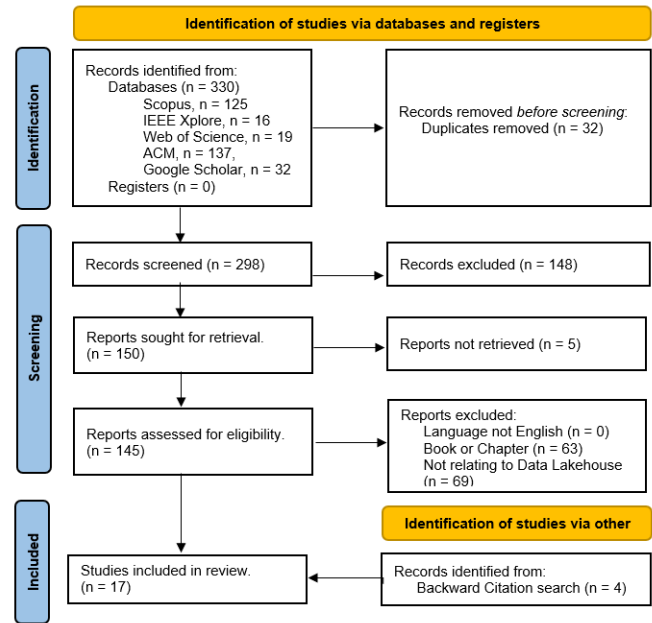


Fig. 1. Process followed during literature screening

B. Definitions of Categorizations

This section outlines the methodology for classifying the literature items into the previously defined categories. This categorization is intended to create a clear focus by organizing the research in a structured manner. Such an approach enhances the ease of analysis, comparison, and summarization, allowing readers to engage with the discussion and grasp the key themes and trends effectively.

• Architecture

A research article on database systems outlines the design and methodology used for data storage, organization, and retrieval, impacting data management efficiency and effectiveness.

• Use Cases

A research article presenting use cases of data lakehouse architecture for time series analysis, categorized by domain.

• Software

A research article that presents technologies utilized in the components of the architecture.

- *Data Schema*

A research article on time series data models in Data Lakehouse implementation, featuring best practices.

The literature items are classified into the proposed categories are given in Table II.

III. LITERATURE REVIEW

A. Concepts and Definitions for Data Lakehouses

An *integrated architecture* should ultimately be the goal of a Data Lakehouse. This concept refers to a single unified data platform that incorporates characteristics and features from both Data Warehouses and Data Lakes [25]. A new definition has been proposed, suggesting that prior definitions tend to overly emphasize functional features while either lacking detail on the underlying architecture or being too abstract to effectively distinguish Data Lakehouses from other data platforms. As a result, these definitions often fail to highlight the specific benefits that Data Lakehouses offer to enterprises. A Data Lakehouse is an integrated data platform that employs the same storage type and data format across various functions, such as reporting, Online Analytical Processing (OLAP), data mining, machine learning, and streaming workloads. Drawing on previous studies [26], eight technical requirements have been established for a data management architecture to be classified as a Data Lakehouse. These requirements are outlined in Section III-B.

The essential components needed to enhance a data lakehouse architecture have been identified [27]. These components encompass flexible big data storage capabilities, tools that support data integration, discovery, metadata management, and analytics from the Data Lake. Moreover, they include structured, clean, and integrated storage, along with components that enable OLAP queries for Business Intelligence (BI) sourced from the Data Warehouse.

B. Architecture

In Table II, we present seventeen items, of which six articles propose a data lakehouse architecture, while two suggest architectures for virtual data warehouses. This section will delve into a detailed discussion and comparison of these architectures, emphasizing their similarities and differences.

A data lakehouse architecture fundamentally comprises at least two essential types of components: *storage components*, which are responsible for the persistent storage of the data, and *processing components*, which facilitate the reading, processing, and writing of data to and from the storage components [25]. Furthermore, the architecture must satisfy eight technical requirements concerning storage, file format, and three analytical workloads: Reporting and OLAP, Machine Learning (ML) and Data Mining, and Streaming. These technical requirements, as outlined by [25], address common challenges observed in Data Lakes, including complex architectures, diverse processing methods, consistency issues, and the necessity for incremental processing. The requirements are as follows,

A data lakehouse architecture designed for the Internet of Things (IoT), illustrated in Figure 2, consists of six layers as

proposed by [23]. Each layer is briefly described as follows: The *Collection Layer* focuses on gathering a variety of IoT data, including images, videos, sensor outputs, and text, from multiple sources. This layer also compresses, validates, and encrypts the data to ensure security and consistency. The *Platform Layer* manages connections to IoT devices, facilitating tasks such as registration, authentication, monitoring, diagnostics, and firmware updates. The *Storage Layer* is responsible for storing data in its native format within a central repository, alongside managing metadata and caching. The *Computing Layer* transforms and processes data using various frameworks, including batch and stream processing. The *Service Layer* employs APIs and interfaces to expose and consume data from the data lake. Finally, the *Application Layer* offers functionalities for end users to interact with the data for business intelligence (BI), reporting, dashboarding, machine learning (ML), and other applications. Given that IoT data generally consists of a time series structure, this architecture has the potential for extension to accommodate broader time series data. While the reference architecture and potential technology stacks have been proposed, the authors have yet to implement, benchmark, and evaluate the performance, efficiency, and impact of IoT Lakehouse solutions across various scenarios and requirements. Consequently, the challenges that may arise during implementation remain undocumented.

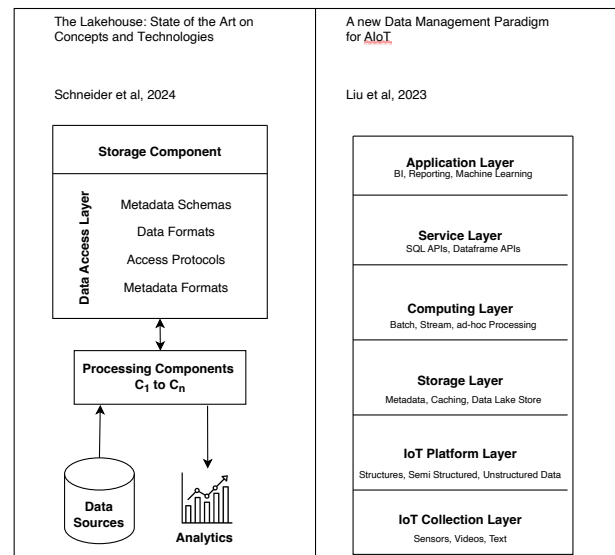


Fig. 2. Data lakehouse architectures proposed by [25] and [23]

An architecture for Biomedical Data was implemented following the architectural design proposed by [6], [7]. The evaluation of this architecture focused on space reduction, performance enhancement, and user adoption rates. Notably, the implementation achieved a tenfold reduction in space requirements, coupled with a one-third improvement in query performance.

A healthcare framework centered on the Findable, Acces-

TABLE II
LIST OF SELECTED LITERATURE ITEMS AND ITS CATEGORIZATION BASED ON CONTENT

Title	Reference	Database	Categorisation
The data warehouse virtualization framework for operational business intelligence	[10]	Scopus	Architecture
Data Warehouse and Data Virtualization Comparative Study	[11]	Scopus	Architecture
Delta lake: high-performance ACID table storage over Cloud Object Stores	[12]	ACM	Use Case, Software
A Lakehouse Architecture for the Management and Analysis of Heterogeneous Data for Biomedical Research and Mega-bio banks	[7]	Backward Search	Architecture, Use Case
An Agile Data Warehouse Virtualization Framework for ROLAP Server	[13]	ACM	Architecture, Use Case
AIOPS Essential to Unified Resiliency Management in Data Lakehouses	[14]	Scopus	Other
From Data Warehouse to Lakehouse: A Comparative Review	[15]	Backward Search	Architecture, Use Case
Meta's next-generation realtime monitoring and analytics platform	[16]	ACM	Use Case
Chrontext: Portable SPARQL queries over contextualised time series data in industrial settings	[17]	Google Scholar	Other
Data Virtualization Enabling Distributed Data Architectures: Data Fabric and Data Mesh	[18]	Scopus	Architecture
Design of Vessel Data Lakehouse with Big Data and AI Analysis Technology for Vessel Monitoring System	[19]	Backward Search	Architecture, Use Case, Data Schema
FAIR Health Informatics: A Health Informatics Framework for Verifiable and Explainable Data Analysis	[20]	Google Scholar	Architecture, Use Case
GeoLake: Bringing Geospatial Support to Lakehouses	[21]	Google Scholar	Architecture, Use Case
Investigating Lakehouse-Backbones for Vehicle Sensor Data	[22]	Scopus	Use Case, Data Schema
IoT Lakehouse: A New Data Management Paradigm for AIoT	[23]	Google Scholar	Architecture
TensorBank: Tensor Lakehouse for Foundation Model Training	[24]	Scopus	Use Case, Data Schema
The Lakehouse: State of the Art on Concepts and Technologies	[25]	Google Scholar	Architecture, Use Case, Software

TABLE III
EIGHT TECHNICAL REQUIREMENTS PROPOSED BY [25]

Same storage type & data format	CRUD for all types of data
Relational data collections	Query language
Consistency guarantees	Isolation & atomicity
Direct read access	Unified batch/stream processing

sible, Interoperable, and Reusable (FAIR) Strategy has been introduced [20]. This framework facilitates the acquisition, management, and processing of both static and dynamic

data, aiming to create a universal clinical schema capable of encompassing various healthcare-related data types, including patient information, clinical data, vaccine details, and related concepts. According to [20], the raw data collection layer is optional, allowing for direct storage in the Data Lakehouse. While the majority of the components in the proposed framework align with existing architectures, the authors suggest three distinct components for processing, each addressing different facets. The *Processing* component is charged with converting data into a standardized format and storing it in the Data Lakehouse, while *Semantic Knowledge Profiling*

focuses on processing the data stored in the Data Lakehouse and integrating information from various sources. Finally, the *Computing Layer* is dedicated to prediction and analysis.

Further, the implementation of data anonymization and scheduled deletions is essential to safeguarding the confidentiality of data collected in the medical field [20]. The framework comprises a four-step onboarding process, as illustrated in Figure 3.

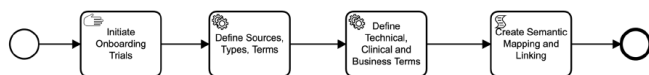


Fig. 3. Dataset on-boarding business process by [20].

The healthcare framework insists that once the dataset is on-boarded, the technical schema should be version-controlled.

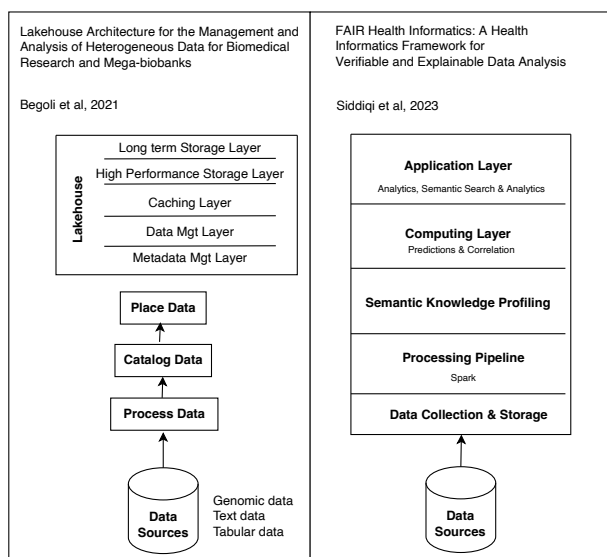


Fig. 4. Data lakehouse architectures proposed by [7] and [20]

A four-layer Data Lakehouse architecture has been proposed for vessel data management, consisting of layers for extraction and ingestion, a Data Lake, a Data Warehouse partitioned into Big Data and AI layers, and an application service layer [19]. The Big Data Layer includes modules for data management and analysis. The architecture for the vessel Data Lakehouse employs a two-tier design [26], which integrates a Data Lake and a Data Warehouse. However, data cannot be directly shared between these two platforms, necessitating data transfer, which can increase architectural complexity and potentially lead to integrity issues. Additionally, there may be latency in the availability of data for analysis.

In their review of data lake architectures, [15] identified several key features essential for incorporation into the Data Lakehouse architecture: Data Access, Data Ingestion, Data Transformation and Loading, Data Storage, and Metadata Management.

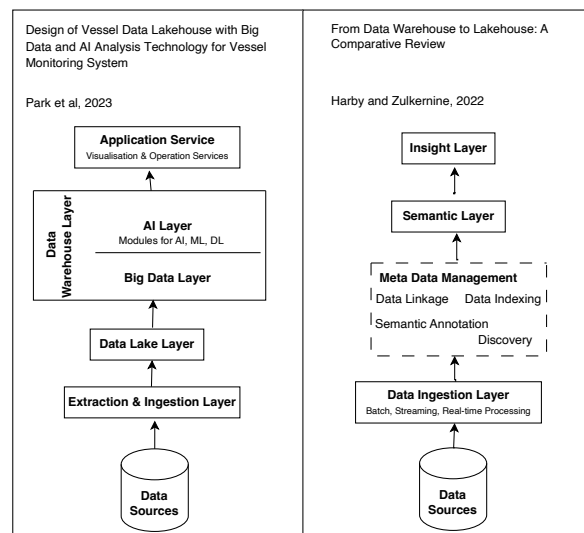


Fig. 5. Data lakehouse architectures proposed by [19] and [15]

An overview of the data lakehouse architectures proposed can be observed in Figures 2, 4, and 5.

In addition to Data Lakehouses, Data Virtualization (DV) has been introduced to address the challenges associated with Data Warehouses and Lakes.

DV facilitates real-time access to data sources, providing an up-to-date view by directly connecting to source systems rather than transferring data to a central repository [18]. It offers a layer of abstraction that conceals aspects of the application, such as data storage and format [11]. The authors observe that while DV enables real-time data access with greater flexibility than a traditional Data Warehouse, there may be concerns regarding data accuracy and validity.

Virtualization plays a crucial role in supporting distributed data architectures, such as Data Fabric and Data Mesh [28]. Consequently, the potential of virtual data warehouses to facilitate these distributed architectures was explored, leading to the evaluation of seven integration patterns based on factors such as ease of use, maintainability, performance, error handling capabilities, reusability, and extensibility [18]. The findings indicate that DV achieved a higher overall score as well as an agility score when compared to Extract-Transform-Load (ETL), Enterprise Service Bus (ESB), Change Data Capture (CDC), Data as a Service (DaaS), and Streaming Data Integration (SDI). Additionally, DV provides essential interoperability, governance, and security for Data Fabric and Data Mesh; however, the authors did not specify the architecture for these distributed concepts.

An Agile BI framework for Relational Online Analytical Processing (ROLAP) was proposed [13]. Query performance was compared between Agile ROLAP and existing models through a real-world BI solution. The results showed that this approach simplifies deployment, is practical, and yields consistent results with traditional methods. The authors concluded

that materialized views outperformed standard views.

C. Data Schema

[22] examined one-dimensional vehicle sensor data, noting that many sensors produce information in this form while considering higher-dimensional measurements (V) for future research. As the data points represent time-based events, each entry includes a timestamp (T) and a sensor ID (SID). The study also addressed the complexities involved in managing asynchronous multivariate time series data by evaluating three different schemata: (1) a schema with columns for T, V, and SID; (2) a schema that organizes data into separate tables for each group of concurrently captured time series sharing a common time axis; and (3) a structure designed for unique SID and T, where measured values are nested, accompanied by two metadata storage strategies (normalization and denormalization). A notable feature of Schema-2 is the inclusion of a lookup table for each vehicle, enabling the database to navigate across multiple tables seamlessly. These schemata are illustrated in Figure 6. The evaluation revealed that Schema-2 offers the most optimal query execution speeds and efficient data storage for specific systems. Furthermore, the majority of systems demonstrated improved performance with a normalization strategy. The evaluation results are summarized in Table IV.

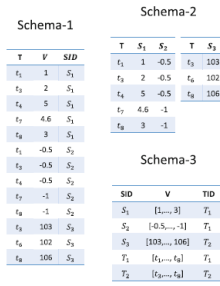


Fig. 6. Three schema developed for evaluation [22]

Further, the performance of the vessel Data Lakehouse was evaluated based on three types of data, which is shown in Figure 7

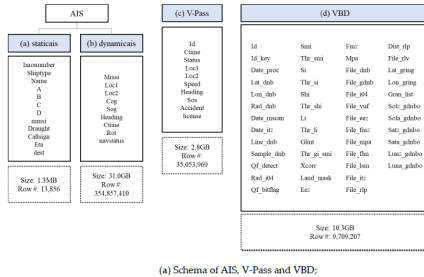


Fig. 7. Vessel data used for evaluation [19]

D. Software

This section reviews software options for implementing data lakehouse architectures. Various literature sources evaluated

technologies to identify the most suitable choices, categorizing them into *Data Storage and Management*, *Processing Tools*, *Management Frameworks*, and *Other*. A total of twenty-one technology combinations were assessed to meet the eight technical requirements outlined in Table III. Ultimately, only three combinations fulfilled these requirements [25]. The best storage solutions utilized Apache Spark for processing, coupled with either the Hadoop Distributed File System (HDFS) or Cloud Object Storage. The frameworks meeting the requirements—particularly for governance and consistency—were Delta Lake, Apache Hudi, and Apache Iceberg.

1) *Data Storage and Management*: In [23], the authors propose using Distributed File Systems or Object Storage solutions like Azure Data Lake Storage (ADLS), Amazon S3, or HDFS for *Data Lake storage*. While they mention various technologies, they do not specify an implementation or the most suitable technology for each architecture layer. Azure Blob Storage [20], [22], [25], HDFS [19], [25], and Amazon S3 [15] were used as storage components in data lakehouse architectures, with Amazon S3 identified as the most cost-effective option compared to Apache Hive, Google Cloud, and HDFS [15].

Three databases and file systems were evaluated for optimal time series data storage and querying [22], including TimescaleDB, InfluxDB, Druid, Hive, Impala, and Spark. Performance was assessed based on data ingestion and retrieval times. This implementation involved three databases—TsDB (v2.8.0), InfluxDB (v2.4.0), and Druid (v24.0.0)—along with three Hadoop systems: SparkSQL (Spark) (v3.3.0), Hive (v3.1.3), and Impala (v4.1.0), all in their default configurations [22]. The findings reveal that schema-3 had the lowest overall ingestion time, with Impala achieving the fastest results, followed by Spark. Additionally, the combination of Schema-2 with Druid, Hive, Impala, and Spark resulted in the smallest increase in storage size. TsDB with Schema-2 excelled in executing high-selectivity queries for both aggregated and unaggregated data. An overview of the results can be found in Table IV.

Additionally, using the Parquet file format reduced storage usage by tenfold compared to CSV, also enhancing performance [7]. This reduction was observed even before utilizing Parquet's built-in compression, indicating further savings could be achieved. Parquet is favored for its integration with Apache Spark [20], while the Delta format proved more efficient on Databricks due to enhanced performance and partitioning, building on Parquet's capabilities [12].

2) *Processing Tools*: Apache Spark has been employed in various proposed architectures [7], [15], [20], [25], [29] as a primary processing tool, with Parquet identified as the preferred file storage format. Additionally, Apache Kafka has been utilized for managing streaming data [20].

A hybrid query engine named "Chrontext" was developed and evaluated to facilitate the execution of SPARQL queries over both static and time series data across different databases. This addresses the challenge of efficiently and portably integrating and querying contextualized time series data in-

TABLE IV
EVALUATION OF TECHNOLOGIES WITH DIFFERENT SCHEMA COMBINATIONS BY [22]

Evaluation of:		Software
2*Data ingestion	Load time	In combination with Schema-3, Impala ; Spark ; Hive ; Druid
	Increase in size	In combination with Schema-2, Hive ; Druid ; Impala ; Spark TsDB is comparable to Hive and Spark in combination with Schema-3
2*Data retrieval	Unaggregated	High selectivity queries: TsDB has best performance in combination with Schema-2 and Schema-1. Impala with Schema-1 has the next-best times, however, is 3x slower Low selectivity queries: Spark with Schema-2 has the optimal speed. Schema-1 is unsuitable to extract a large amount of data.
	Aggregated	High selectivity queries: TsDB with Schema-2 has the optimal speed followed by TsDB with Schema-3 Low selectivity queries: TsDB with Schema-3 has the optimal speed followed by Spark with Schema-2

dustrial environments [17]. Implemented in Rust and utilizing the Apache Arrow-based Polars library, Chronotext achieves high performance. Chronotext meets five key requirements: it supports context-based extraction via SPARQL, ensures high throughput and low latency for time series extraction, integrates with established data science and engineering tools, provides query portability, and allows for computation offloading to enhance performance. Its implementation was assessed against Ontop, a leading virtual knowledge graph system, using a synthetic dataset based on industrial standards. Chronotext significantly outperformed Ontop, achieving query execution times that were 10 to 85 times faster, alongside considerably lower memory consumption. Given that the current solution is confined to a single time series database, the authors plan to extend it by enabling support for multiple time series backends and their associated metadata. This strategy aims to eliminate vendor lock-in and foster semantic interoperability, allowing other teams to comprehend and reuse the analytical dataset.

3) *Management Frameworks*: The key management frameworks discussed include Delta Lake, Apache Iceberg, and Apache Hudi [23]. Their utilization is further verified through various implementations highlighted in the research considered. These data management frameworks incorporate essential features for effective data governance, such as metadata management, time travel, streaming and batch processing capabilities, and audit logs, among others. As previously noted, [25] also corroborates that these three frameworks meet the eight technical requirements proposed in their analysis. An evaluation of these three systems was conducted based on their performance and features using the open-source benchmark suite LHBench [29]. They were selected for their diverse data management capabilities and their growing popularity as open storage systems. The evaluation focused on three key areas: overall performance, the efficacy of various data ingestion strategies, and the performance of different metadata access strategies during query planning with Apache Spark on AWS EMR, utilizing AWS S3 for data storage. EMR was chosen for this evaluation because it supports all three frameworks without being specifically optimized for anyone, which the authors believe ensures a fair assessment. A summary of the evaluation results can be found in Table V.

The implementation of Delta Lake, Apache Atlas, and

Unity Catalog for metadata management, along with Alluxio for cache management, was proposed by [23]. Additionally, Apache Hudi was utilized within the Health Informatics framework, with its selection based on its time travel capabilities [20]. Delta Lake, however, emerged as the more popular choice in data lakehouse implementations [7], [15], [19]. [19] plans to extend their current Vessel Lakehouse implementation to incorporate Delta Lake, aiming to manage large volumes of image data in future work. This popularity may be attributed to its high adoption rate and versatility with various data processing tools such as Apache Spark, Hive, Presto, and Redshift, as well as its compatibility with multiple Cloud Object Stores, including Amazon S3, Google Cloud Storage, and Azure Blob Storage [12]. In the data warehousing layer, Apache Impala—a query processing engine—and Apache Kudu—an open-source distributed data storage engine—were also employed [19]. Apache Kudu is recognized for its primary key support, enabling millisecond-level random access, in contrast to other columnar storage formats, and supports both OLAP and Online Transaction Processing (OLTP). The performance of the Data Lakehouse was assessed in comparison to PostgreSQL, utilizing three types of vessel-specific data (static AIS, dynamic AIS, and V-Pass). The findings revealed that the data lakehouse implementation was 88

4) *Other Software Types*: Azure IoT Hub, AWS IoT Core, Google Cloud IoT Core, and IBM Watson IoT Platform are identified as IoT platforms that can be integrated into a data lakehouse architecture [23]. These IoT platforms facilitate the collection of data from devices and drive business transformation by acting as middleware to manage all interactions between user applications and remote devices. They also seamlessly integrate with cloud platforms for storage, utilize big data technologies to handle and process large datasets, connect with ERP systems to optimize business operations, and much more.

E. Use Cases

1) *Health*: [20] introduced three specific use cases for the proposed framework. Firstly, the framework facilitates data accessibility for various stakeholders, including government organizations and NGOs, enabling them to conduct research and analysis. According to the authors, stakeholders can onboard datasets by defining both technical and conceptual schemas, as well as providing the necessary means for the

TABLE V
RESULTS OF PERFORMANCE EVALUATION BY [29]

	Delta Lake	Apache Iceberg	Apache Hudi
Overall load times	~12.7 mins for 3 TB of TPC-DS data	~14.9 mins for 3 TB of TPC-DS data	10x slower, as it is optimised for keyed upserts, not bulk data ingestion
Overall query performance	1.7x faster than Hudi 1.4x faster than Iceberg	Iceberg uses a custom-built Parquet reader in Spark that is significantly slower than the default Spark reader used by Delta Lake and Hudi	Difference is based on data reading time. Hudi partitions using smaller file sizes, which reduces efficiency in columnar compression and requires large table scans
Merges	Only Copy-on-Write (CoW) Competitive merge due to generating fewer files, faster scan, and an optimised MERGE command.	Both Merge-on-Read (MoR) and CoW MoR: 1.3x faster than Hudi	Both MoR and CoW Post-merge queries: 3.2x faster than Iceberg
Metadata processing	Runs on Spark Is 7-12x faster for larger datasets	Runs on a single node Is more performant for small datasets	

framework to access the data, whether through APIs or data storage solutions. Secondly, this framework allows for the integration of data with other publicly available datasets while safeguarding the privacy of the imported data. Lastly, it has the capability to identify additional datasets pertinent to the domain of interest within a specified timeframe. At Oak Ridge National Laboratory (ORNL), teams utilizing data managed by the Data Lakehouse are engaged in open-ended data science discovery tasks. They conduct exploratory data analysis to pinpoint anomalies or intriguing phenomena, which can then lead to further exploratory investigations. These teams utilize standard data science and machine learning libraries to access and analyze relevant data within the Data Lakehouse, adhering to established data access permissions [7]. Furthermore, biomedical research at ORNL is conducted within virtual machines, allowing researchers to access authorized data from the Data Lakehouse. This research typically operates under Institutional Review Board (IRB) regulations or operational protocols, and the virtual machines and user accounts are designed to ensure compliance with these access controls [7].

2) *IoT*: [23] highlights that the IoT data lakehouse architecture proposed can accommodate a diverse range of use cases. These include but are not limited to, quality control, remote monitoring, asset optimization, and fleet management within Industrial IoT, as well as traffic management, smart parking, smart lighting, smart metering, smart waste management, and smart surveillance in Smart Cities. Data lakehouses offer a robust solution by enabling the storage of multiple data types in their native formats, allowing for flexible processing as needed. Although several potential use cases are mentioned, the article does not delve into a specific use case that could be explored in greater depth. [22] conducted a benchmarking study on data structures and metadata concepts to identify an optimal system for storing vehicle sensor data. Their findings indicate that the data structures must be optimized to achieve peak performance and outline an optimal schema that should be adhered to for sensor data. This schema is further elaborated upon in Section III-C.

3) *Other Domains*: The healthcare framework developed by [20] utilizes data lakehouse technology and offers various abstractions, making it adaptable for a range of applications. This versatility could extend to use cases such as weather monitoring, traffic and air management, and other scenarios

involving complex event processing [20].

F. Other

A data management solution designed specifically for Data Lakehouses, named "Geo Lake," has been introduced to support geospatial data management [21]. This solution provides native capabilities for storing geometry values within data lakehouse tables, facilitating flexible querying and processing of geospatial tables through Spatial SQL. Key features include spatial partitioning and spatial predicate pushdown integrated into the data lakehouse architecture, which contributes to accelerated geospatial queries via efficient data skipping and optimized data layout. A Parquet file format has been developed explicitly for rapid scanning of encoded geometry values. Performance evaluations conducted on Geo Lake, in comparison with GeoParquet and GeoParquet (bbox), have demonstrated reading speed enhancements of up to 20 times in specific scenarios, using real-world datasets. However, due to the specialized nature of this solution concerning geospatial data, a detailed discussion will not be provided here. The core code for this solution has been made available as open source on GitLab for further reference.

In contrast to our primary focus on architectures better suited for time series data in tabular formats, a data lakehouse architecture termed "TensorBank," designed for data organized in tensors for Foundation Model Training, has been presented [24]. TensorBank largely adheres to the general data lakehouse architecture but incorporates certain modifications to ensure compatibility with other data lakehouses. One notable alteration is the adoption of ZARR as the storage format, which is specifically tailored for multi-dimensional tensor data, rather than utilizing Apache Parquet or Avro.

The significance of managing hardware resources within data lakehouses to achieve Data Lakehouse Recovery Point Objectives (RPO) with high precision has been emphasized [14]. RPO failures primarily arise from inadequate network or storage bandwidth. To address this issue, an ensemble of machine learning techniques combined with time series analysis has been proposed. This ensemble aims to predict potential violations of the system's RPO and estimate their severity. Moreover, the model is designed to identify the type of resources that could be augmented to avert the anticipated violation.

IV. DISCUSSION

While it can be noted that each research article presents slightly different architectures tailored to their specific use cases, they share common elements in their proposed layers or components. The presence of components for Data Collection and Ingestion, Processing, Storage, and Application Services can be observed in many of these architectures [15], [20], [23], [25].

Additionally, the metadata layer of a Data Lakehouse plays a critical role in its overall functionality. This layer provides essential management features such as transactions, time travel, and data quality enforcement, making it advantageous for organizations already using a Data Lake. The importance of metadata management is evident in the architectures presented [7], [15], [23], [25]. A detailed discussion of the technologies utilized for metadata management can be found in Section III-D.

Based on the insights gained from the evaluation and literature review, the conceptual architecture illustrated in Figure 8 is proposed.

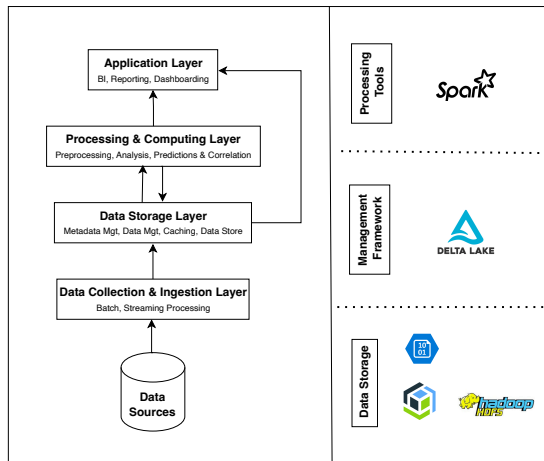


Fig. 8. Proposed conceptual architecture and recommended technologies based on review

In addition, utilizing HDFS or Cloud Object Stores for data storage, Apache Spark for data processing, and Delta Lake as a management framework constitutes a highly effective combination of technologies for a high-performing and scalable data lakehouse architecture. Based on the findings of various studies by Begoli et al. (2021), Vox et al. (2023), Park et al. (2023), and Liu et al. (2023), these software stacks are particularly recommended for handling time series data, as the implementations presented were grounded in time series data across multiple domains. Notably, TsBD is identified as the most optimal solution for time series data, specifically when utilized in conjunction with Schema-2 as defined by Vox et al. (2023). Apache Spark demonstrated strong performance metrics, including reduced load times, effective data management, and efficient execution of low-selectivity queries. Conversely,

it was noted that Schema-1 was inadequate for managing large datasets, while Schema-2 exhibited optimal capability for query execution, and Schema-3 excelled in ingestion speed.

While several research initiatives have assessed the performance and scalability of various software technologies, many evaluations have employed default configurations. To ascertain the most suitable technology for diverse use cases, it is crucial to evaluate these technologies under optimal configurations. Consequently, the inquiry arises: how can each system be optimized to fulfill specific data and analytical requirements? Which configurations can facilitate optimal scalability within the data lakehouse architecture? Furthermore, there remains a pressing need for research into cost optimizations to ensure affordability in the realms of data storage, analysis, and artificial intelligence workloads.

Moreover, a notable research gap exists regarding data organization, governance, and security within data lakehouses. Current investigations into the optimization of data models for time series data are limited and warrant further exploration in future studies. There is also an open research opportunity to enhance modern data platforms, such as data lakehouses, to facilitate self-service analytics, thereby enabling data analysts and scientists to independently locate, access, prepare, and utilize their data. Addressing this gap necessitates the development of frameworks for user data preparation and pre-processing, while simultaneously considering privacy and access control, as users presently rely on their own infrastructures or data engineers for these essential tasks.

V. CONCLUSION

In conclusion, the emergence of the Data Lakehouse addresses the challenges and limitations associated with traditional Data Warehouses and Data Lakes, particularly concerning high costs, integrity issues, and inflexible schemas. A systematic literature review reveals that proposed data lakehouse architectures for various time series use cases share common components related to data extraction, ingestion, storage, processing, and application services. Additionally, most implementations rely on a limited set of technologies for data storage, management, and processing. Therefore, identifying and implementing optimal configurations is crucial to meeting the specific data and analysis needs of different scenarios. Cost considerations are also vital for the practical deployment of data lakehouse architectures, especially given the demands of extensive data analysis and AI workloads. Future research should focus on developing detailed optimization strategies for diverse systems while exploring cost-effective solutions to enhance the accessibility and efficiency of Data Lakehouses. Furthermore, effective governance and security measures must be prioritized to ensure comprehensive oversight. By addressing these facets, organizations can better leverage the capabilities of data lakehouse architectures for managing and analyzing large-scale data effectively.

ACKNOWLEDGMENT

This work is funded by the German Ministry of Research and Education (BMBF) under grant number 03F0888A: M-VRE: The MOSAiC - Virtual Research Environment.

REFERENCES

- [1] V. Mayer-Schönberger and K. Cukier, *Big data: A revolution that will transform how we live, work, and think*. Boston, MA: Houghton Mifflin Harcourt, 2013.
- [2] M. Pohl, C. Haertel, D. Staegemann, and K. Turowski, "Data Science Methodology," in *Encyclopedia of Data Science and Machine Learning*, J. Wang, Ed. IGI Global, Oct. 2022, pp. 1201–1214. [Online]. Available: <https://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/978-1-7998-9220-5.ch070>
- [3] M. Volk, D. Staegemann, N. Jamous, M. Pohl, and K. Turowski, "Providing Clarity on Big Data Technologies: The BDTonto Ontology," *International Journal of Intelligent Information Technologies*, vol. 16, no. 2, pp. 49–73, Apr. 2020. [Online]. Available: <http://services.igi-global.com/resolvedoi/resolve.aspx?doi=10.4018/IJIT.2020040103>
- [4] F. Nargesian, E. Zhu, R. J. Miller, K. Q. Pu, and P. C. Arocena, "Data lake management: challenges and opportunities," in *Proceedings of the VLDB Endowment* 12, 2019.
- [5] O. Azeroual, J. Schöpfel, D. Ivanovic, and A. Nikiforova, "Combining data lake and data wrangling for ensuring data quality in crisis," 2022.
- [6] M. Armbrust, A. Ghodsi, R. Xin, and M. Zaharia, "Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics," 2021.
- [7] E. Begoli, I. Goethert, and K. Knight, "Alakehouse architecture for the management and analysis of heterogeneous data for biomedical research and mega-biobanks," 2021.
- [8] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," vol. 2, 01 2007.
- [9] M. J. Page, D. Moher, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and J. E. McKenzie, "Prisma 2020 explanation and elaboration: updated guidance and exemplars for reporting systematic reviews," *BMJ*, vol. 372, 2021. [Online]. Available: <https://www.bmj.com/content/372/bmj.n160>
- [10] F. Farooq, "The data warehouse virtualization framework for operational business intelligence," *Expert Systems*, vol. 30, 11 2013.
- [11] A. H. Mousa and N. Shiratuddin, "Data warehouse and data virtualization comparative study," in *2015 International Conference on Developments of E-Systems Engineering (DeSE)*, 2015, pp. 369–372.
- [12] M. Armbrust, T. Das, L. Sun, B. Yavuz, S. Zhu, M. Murthy, J. Torres, H. van Hovell, A. Ionescu, A. Łuszczak, M. undefinedwitakowski, M. Szafranski, X. Li, T. Ueshin, M. Mokhtar, P. Boncz, A. Ghodsi, S. Paranjpye, P. Senster, R. Xin, and M. Zaharia, "Delta lake: high-performance acid table storage over cloud object stores," *Proc. VLDB Endow.*, vol. 13, no. 12, p. 3411–3424, aug 2020. [Online]. Available: <https://doi.org/10.14778/3415478.3415560>
- [13] A. A. Menolli, R. G. Coelho, G. C. Silva, and E. Barbosa, "An agile data warehouse virtualization framework for rolap server," in *Proceedings of the XVII Brazilian Symposium on Information Systems*, ser. SBSI '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: <https://doi.org/10.1145/3466933.3466980>
- [14] R. Jin, P. Muench, V. Deenadhayalan, and B. Hatfield, "Aiops essential to unified resiliency management in data lakehouses," in *2022 IEEE International Conference on Big Data (Big Data)*, 2022, pp. 4777–4781.
- [15] A. Harby and F. Zulkernine, "From data warehouse to lakehouse: A comparative review," 12 2022.
- [16] S. Harizopoulos, T. Hopper, M. Mo, S. S. Chandrasekaran, T. Chen, Y. Cui, N. Ganesh, G. Helmling, H. Pham, and S. Wong, "Meta's next-generation realtime monitoring and analytics platform," *Proc. VLDB Endow.*, vol. 15, no. 12, p. 3522–3534, aug 2022. [Online]. Available: <https://doi.org/10.14778/3554821.3554841>
- [17] M. Bakken and A. Soylu, "Chronotext: Portable sparql queries over contextualised time series data in industrial settings," *Expert Syst. Appl.*, vol. 226, no. C, sep 2023. [Online]. Available: <https://doi.org/10.1016/j.eswa.2023.120149>
- [18] M. Akermi, M. A. hadj Taieb, and M. B. Aouicha, "Data virtualization enabling distributed data architectures: Data fabric and data mesh," vol. 15, p. 12, Jan. 2023. [Online]. Available: <https://cspub-ijcism.org/index.php/ijcism/article/view/511>
- [19] S. Park, C.-S. Yang, and J. Kim, "Design of vessel data lakehouse with big data and ai analysis technology for vessel monitoring system," *Electronics*, vol. 12, p. 1943, 04 2023.
- [20] M. H. Siddiqi, M. Idris, and M. Alruwaili, "Fair health informatics: A health informatics framework for verifiable and explainable data analysis," *Healthcare*, vol. 11, no. 12, 2023. [Online]. Available: <https://www.mdpi.com/2227-9032/11/12/1713>
- [21] Y. Zhang, B. Peng, Y. Du, and J. Su, "Geolake: Bringing geospatial support to lakehouses," *IEEE Access*, vol. PP, pp. 1–1, 01 2023.
- [22] C. Vox, D. Broneske, J. Piewek, J. Feigel, and G. Saake, "Investigating lakehouse-backbones for vehicle sensor data," in *Database and Expert Systems Applications*, C. Strauss, T. Amagasa, G. Kotsis, A. M. Tjoa, and I. Khalil, Eds. Cham: Springer Nature Switzerland, 2023, pp. 243–258.
- [23] G. Liu, Z. Pang, J. Zeng, H. Hong, Y. Sun, M. Su, and N. Ma, "Iot lakehouse: A new data management paradigm for smart cities," in *Big Data – BigData 2023: 12th International Conference, Held as Part of the Services Conference Federation, SCF 2023, Honolulu, HI, USA, September 23–26, 2023, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2023, p. 34–47. [Online]. Available: https://doi.org/10.1007/978-3-031-44725-9_3
- [24] R. Kienzler, J. Schmude, N. Simumba, B. Blumenstiel, M. Freitag, D. Kimura, Z. Nagy, M. Behrendt, H. Hamann, S. Muckavilli, and D. Civitarese, "Tensorbank: Tensor lakehouse for foundation model training," in *2023 IEEE International Conference on Big Data (BigData)*. Los Alamitos, CA, USA: IEEE Computer Society, dec 2023, pp. 3350–3354. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/BigData59044.2023.10386912>
- [25] J. Schneider, C. Gröger, A. Lutsch, H. Schwarz, and B. Mitschang, "The lakehouse: State of the art on concepts and technologies," *SN Computer Science*, vol. 5, no. 5, p. 449, Apr 2024. [Online]. Available: <https://doi.org/10.1007/s42979-024-02737-0>
- [26] J. Schneider, C. Gröger, A. Lutsch, H. Schwarz, and B. Mitschang, "Assessing the lakehouse: Analysis, requirements and definition," 01 2023.
- [27] B. Shiyal, *Modern Data Warehouses and Data Lakehouses*. Berkeley, CA: Apress, 2021, pp. 21–48. [Online]. Available: https://doi.org/10.1007/978-1-4842-7061-5_2
- [28] X. Li, M. Yang, X. Xia, K. Zhang, and K. Liu, "A distributed data fabric architecture based on metadata knowledge graph," in *2022 5th International Conference on Data Science and Information Technology (DSIT)*, 2022, pp. 1–7.
- [29] P. Jain, P. Kraft, C. Power, T. Das, I. Stoica, and M. A. Zaharia, "Analyzing and comparing lakehouse storage systems," in *Conference on Innovative Data Systems Research*, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259267242>